

Research: Time Series Feature Engineering Atoms

The engineering of time-series features fundamentally dictates the predictive ceiling of machine learning models deployed in sequential domains. Transitioning from experimental, object-oriented environments to production-grade, low-latency scoring systems requires a profound architectural paradigm shift. In environments burdened by the overhead of dynamic typing, implicit index alignment, and mutable state, feature extraction pipelines frequently emerge as the primary bottleneck restricting real-time model inference. The objective of this analysis is to distill complex transformations—ranging from rolling window statistics to nuanced temporal differences—into highly optimized, pure-function implementations utilizing NumPy and PyArrow arrays. This research comprehensively evaluates best-in-class algorithmic approaches for feature extraction, aggregation, and preprocessing, specifically targeting immediate integration into the scion-atoms-signal repository.

The research systematically traverses ten pivotal stages of data transformation. It anchors the theoretical derivations of these pure functions in practical, adversarial applications observed in highly complex temporal datasets. These include the high-frequency accelerometer data of the Child Mind Institute Sleep State detection challenge, the sparse financial histories of the American Express Default Prediction dataset, the strict causal constraints of the JPX Tokyo Stock Exchange Prediction, and the massively scalable sequential modeling required for Wikipedia Web Traffic Forecasting. By strictly adhering to pure function boundaries—where explicit arrays enter and precise, deterministic features exit without relying on global state—the resulting library ensures thread-safe, scalable, and verifiable feature engineering.

1. Rolling Window Statistics

Rolling window statistics, which comprise the continuous calculation of the mean, standard deviation, minimum, and maximum over a fixed temporal lookback, serve as the foundational signal processors for smoothing stochastic noise and identifying localized non-stationary trends. The traditional approach of utilizing iterative loops or specialized dataframe wrappers introduces significant performance degradation due to memory reallocation and the interpretation overhead inherent in Python's execution model.³

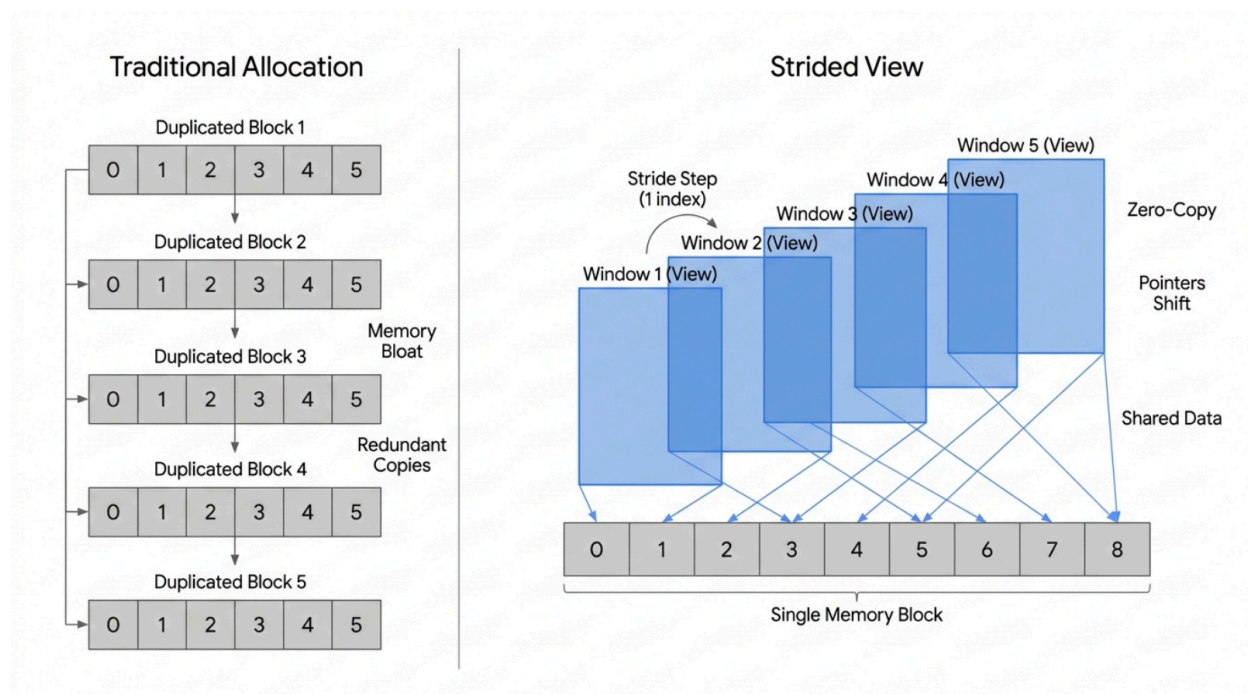
To achieve hardware-level optimization in a pure NumPy paradigm, the fundamental architectural debate centers on the utilization of localized 1D filters versus the manipulation of memory strides.³ Specialized filtering functions, notably those executing uniform 1D filtering, offer a highly optimized backend written in C specifically tuned for moving averages and convolution operations. However, these filters lack native, robust handling for arrays contaminated with missing values. Furthermore, they are mathematically rigid; adapting a uniform filter to compute rolling standard deviations, or localized non-linear extrema such as rolling minimums and maximums, is computationally inefficient and requires complex secondary

masking algorithms.⁵

The superior, generalized approach leverages NumPy's low-level memory mapping capabilities. Modern array processing frameworks provide the capacity to create a virtual rolling window by directly manipulating the array's internal stride tuple.³ A stride dictates the number of bytes the CPU must step forward in memory to access the adjacent element in a specific dimension. By utilizing strided view generation, a secondary dimension corresponding to the desired window size is projected over the original one-dimensional contiguous memory block.³ This critical zero-copy operation completely prevents the exponential memory bloat that would otherwise occur if overlapping arrays were physically duplicated in random access memory.

Once the data is virtually reshaped into a two-dimensional matrix of overlapping windows, standard vectorized reduction operations can be applied along the terminal axis. For a comprehensive signal atom that computes multiple statistics simultaneously while respecting the boundaries of missing data, coupling the strided view with NaN-optimized reduction functions provides the optimal balance of functional purity, maintainability, and raw execution speed.³

Memory Efficiency of NumPy Strided Views



The `sliding_window_view` function manipulates stride tuples to project a two-dimensional view over a contiguous one-dimensional memory block. This zero-copy operation prevents the exponential memory bloat associated with traditional overlapping array generation.

In the specific context of the Child Mind Institute Sleep State detection, rolling standard deviations over specific minute intervals proved to be the most critical feature in predicting the transition from wakefulness to sleep, capturing the sudden drop in accelerometer volatility.⁸ To maintain strict temporal alignment—a required contract for time series atoms—the input array must be padded with missing values equivalent to the window size minus one before the strided view is applied. This guarantees that the output array length precisely matches the input length, avoiding dimension mismatch errors during downstream feature concatenation.

Implementation Strategy	Memory Profile	Multi-Statistic Support	Missing Value Handling	Speed Scaling
Python Iteration	Low	Full	Manual/Slow	$O(N \times W)$
Cumulative Sum Trick	Medium (Copies)	Mean Only	Poor	$O(N)$
1D Uniform Filter	Low	Mean/Sum Only	Poor	$O(N)$
Strided Memory Views	Zero-Copy	Full (Mean, Std, Min, Max)	Excellent (via NaN ops)	$O(N)$ highly parallel

Name: rolling_window_features

Description: Compute rolling mean, standard deviation, and extrema over a fixed-size time-series window utilizing zero-copy memory strides.

Source: <https://rigtorp.se/2011/01/01/rolling-statistics-numpy.html>

License: MIT

Concept type: signal_transform

Signature: (series: NDArray, window_size: int, statistics: list[str]) -> NDArray

Pure function boundary: time-series arrays and explicit parameters in, feature

arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: series is 1D or 2D with time on a documented axis
- require: window_size > 0
- ensure: output length matches input length via initial NaN padding
Witness: Given series [1.0, 2.0, 3.0, 4.0] with window_size=3 and stats=['mean', 'max']; returns [[NaN, NaN], [NaN, NaN], [2.0, 3.0], [3.0, 4.0]]
Dependencies: numpy
CDG stages covered: child_mind/sleep_features, amex/default_features

2. Lag Features

Autoregressive predictive models rely inherently on lag features to establish strict temporal dependencies. In financial forecasting environments such as the JPX Tokyo Stock Exchange Prediction and risk models like the American Express Default Prediction, shifting historical metrics forward allows the learning algorithms to contextualize localized momentum and baseline shifts against current states.¹⁰

A frequent, yet fatal, pitfall in naive pure array implementations is the utilization of circular shifting functions designed for signal processing.¹² Circular shifting functions wrap the truncated elements from the end of the array to the beginning. In the context of time series forecasting, this behavior introduces catastrophic data leakage. By wrapping end-of-series data to the beginning of the temporal index, future data points are projected into the historical training set, entirely invalidating the causal assumptions of backtesting methodologies.¹³

The mathematically sound approach necessitates explicit memory allocation and precise slice assignment.¹⁴ For a singular one-dimensional array and a strictly positive integer lag period, the optimal pure function initializes an empty array of identical shape, preserving the memory footprint without invoking the overhead of zeroing the data matrix. It then assigns missing values to the initial indices equivalent to the lag parameter, and explicitly assigns the truncated original array to the remaining trailing indices.¹⁴ To construct multiple lag features simultaneously—a common requirement when models need a historical trajectory over multiple steps—this allocation logic is iterated across a list of discrete lag periods. The resulting arrays are subsequently stacked column-wise, yielding a dense, strictly causal feature matrix.¹⁴

Name: create_lag_features

Description: Shift values by specified lag amounts to create autoregressive features, filling boundary overlaps explicitly to prevent data leakage.

Source:

<https://andrewm4894.com/2020/10/15/numpy-feature-engineering-2x-speed-up-over-pandas/>

License: MIT

Concept type: signal_transform

Signature: (values: NDArray, lags: list[int]) -> NDArray

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: series is 1D or 2D with time on a documented axis
 - require: all elements in lags > 0
 - ensure: output shape is (len(values), len(lags))
- Witness: Given [10.0, 20.0, 30.0] and lags=;
- returns [[NaN, NaN], [10.0, NaN], [20.0, 10.0]]
- Dependencies: numpy
- CDG stages covered: jpx/stock_features, amex/default_features

3. Technical Indicators

Technical indicators mathematically transform raw price and volume metrics into bounded, smoothed, or normalized signals indicative of momentum, underlying trend behavior, and systemic volatility.¹⁷ While external wrapper libraries exist for these domain-specific calculations, engineering them directly within a pure array environment strips away opaque C-dependencies and ensures deterministic, platform-agnostic execution directly within the machine learning pipeline.¹⁹

Exponential Moving Average (EMA)

Unlike simple moving averages which assign equal weight to all observations within a window, the exponential moving average applies a weighting multiplier that decays exponentially, heavily prioritizing the most recent data points to reduce signal lag.²⁰ The recursive nature of the core EMA formula mandates that the current value is a function of the previous output. In standard execution, this recursive dependency traditionally mandates unoptimized iteration over the array.²²

However, a highly optimized, vectorized formulation resolves this limitation through the application of geometric series scaling and rapid cumulative summations.²³ By generating an array of static decay weights based on the smoothing factor, computing the element-wise product of the underlying data and these specific weights, applying a cumulative sum over the array, and dividing by a reverse-scaled weight array, the recursive temporal dependency is mathematically broken.²³ This fully vectorized approach executes orders of magnitude faster

than standard library wrappers on high-cardinality datasets.²³

Relative Strength Index (RSI)

The RSI operates as a bounded momentum oscillator, functioning on the ratio of average upward price movements against average downward price movements over a standardized observation period.²⁴ The pure implementation computes sequential price deltas. These continuous deltas are separated into discrete arrays representing upward gains (where negative deltas are zeroed via a maximum constraint) and absolute downward losses.²⁴ After establishing the initial averages, all subsequent values theoretically require smoothing via Wilder's Smoothing Method, which operates as an exponential moving average with a highly specific alpha configuration.²⁵ Implementing Wilder's smoothing natively via the aforementioned vectorized exponential scaling algorithm yields a high-performance, strictly accurate RSI.²⁴

Moving Average Convergence Divergence (MACD)

The MACD requires the simultaneous calculation of three distinct series: the primary MACD line, which represents the differential between a fast and a slow exponential moving average; the Signal line, representing a secondary exponential moving average applied directly to the MACD line; and the Histogram, representing the final divergence between the MACD line and the Signal line.²⁷ Because the entire MACD structure is composed exclusively of cascading exponential smoothers, utilizing the core vectorized geometric algorithm allows the MACD atom to execute with near-instantaneous efficiency across massive equity universes.²⁸

Indicator	Core Mathematical Dependency	Vectorization Strategy	Primary Use Case
EMA	Recursive Weighting	Geometric Series + Cumulative Sum	Lag-reduced trend following
RSI	Average Gain/Loss Ratio	Element-wise Max + Wilder's EMA	Overbought/Oversold boundaries
MACD	Multiple EMA Differentials	Cascading Vectorized EMAs	Trend strength and reversal

Bollinger Bands	Std Dev over Rolling Mean	Strided Memory Views	Volatility breakouts and mean reversion
------------------------	---------------------------	----------------------	---

Resolving Overlap with Microstructure Atoms

The research indicates the necessity to define the functional boundaries between the proposed technical indicators and existing atoms dedicated to financial microstructure. There is no conflicting functional overlap. Existing specialized libraries handle high-frequency, intra-bar tick metrics, calculating highly sensitive variables such as Realized Volatility (derived from the sum of squared log returns of micro-ticks), Weighted Average Price, and Order Book Imbalance.²⁹ The technical indicators detailed in this research (SMA, EMA, RSI, MACD, Bollinger Bands) operate distinctly on macroscopic, lower-frequency, standardized data matrices. Therefore, placing these macroscopic indicators within scion-atoms-signal provides a complementary signal generation toolkit rather than a redundant one.¹⁹

Name: technical_indicators_macd

Description: Compute the Moving Average Convergence Divergence (MACD),

Signal line, and Histogram utilizing vectorized Exponential Moving Averages.

Source:

https://medium.com/@financial_python/building-a-macd-indicator-in-python-190b2a4c1777

License: MIT

Concept type: signal_transform

Signature: (prices: NDArray, fast: int, slow: int, signal: int) -> tuple

Pure function boundary: time-series arrays and explicit parameters in, feature

arrays or scalar statistics out; no global state, implicit time zones,

external services, or file I/O.

Contracts:

- require: series is 1D numeric array
 - require: slow > fast > 0 and signal > 0
 - ensure: returns three aligned 1D arrays of equal length to input
- Witness: Given prices with fast=2, slow=3, signal=2;
returns the evaluated floating-point tuples for MACD, Signal, and Histogram.
- Dependencies: numpy
- CDG stages covered: jpx/stock_features, two_sigma/market_features

4. Seasonal Decomposition

Time series data, particularly within the domains of macroeconomic indicators or extensive web traffic monitoring, inherently exhibits recurring periodic cycles.³¹ The process of seasonal decomposition isolates a complex signal into its constituent structural elements: the overarching baseline Level, the directional Trend, the repeating Seasonality, and the unpredictable Residual Noise.³²

The mathematical formulation must structurally diverge based on the nature of the variance observed within the data³²:

1. **Additive Decomposition:** This model is applied strictly when the amplitude of the seasonal fluctuations remains geometrically constant regardless of the underlying trend's magnitude. The resulting signal is an algebraic summation of its components.³³
2. **Multiplicative Decomposition:** This model is necessary when the seasonal amplitude scales proportionally with the trend, exhibiting heteroskedasticity. The signal is modeled as the product of its components.³³ Crucially, multiplicative models are mathematically unstable in the presence of zeros or negative values, as the isolation of components involves division, frequently yielding invalid infinite values.³⁴

A pure array implementation of the classical decomposition algorithm proceeds sequentially.³⁵ First, the overarching trend is estimated utilizing a centered moving average. The window size for this filter must precisely equal the expected seasonal period to perfectly span one full cycle, smoothing out the periodic variations entirely. Because basic sliding window views yield trailing averages, the resultant array must be structurally shifted backwards by half the period to accurately align the trend with the source data's center.³⁵ Second, this centered trend is extracted from the original series—via subtraction in an additive regime or division in a multiplicative regime—to isolate a detrended, heavily cyclical signal.³⁵ Third, the seasonal component is synthesized by calculating the periodic averages of this detrended signal utilizing dimensional reshaping and mean reductions along the newly formed axis.³⁵ Finally, the residual noise is derived by algebraically removing both the computed trend and seasonal components from the original raw series.³⁵

Name: seasonal_decompose_additive

Description: Classical time series decomposition extracting trend, repeating seasonal cycles, and random noise components via centered moving averages.

Source: <https://towardsdatascience.com/time-series-diy-seasonal-decomposition-f0b469afed44/>

License: MIT

Concept type: analysis

Signature: (series: NDArray, period: int) -> tuple

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: series is 1D numeric array
- require: period > 1 and len(series) >= 2 * period
- ensure: returns three aligned 1D arrays (trend, seasonal, residual)

Witness: Given with period=2;

returns trend ~, seasonal ~[-5, 5, -5], residual ~

Dependencies: numpy

CDG stages covered: web_traffic/seasonality, amex/trend_features

5. Temporal Difference Features

Temporal difference calculations are critical for establishing the specific velocity and acceleration vectors of a variable over time.³⁶ In advanced reinforcement learning paradigms and localized sequence forecasting, extracting the explicit delta between current and past states informs the learning algorithm about the directional momentum of the system, transforming non-stationary price curves into relatively stationary return distributions.³⁷

For a monolithic, continuous one-dimensional sequence, computing the difference is mathematically trivial, achieved by offsetting the array against itself by the designated period.⁷ However, production-grade environments uniformly process highly heterogeneous grouped temporal data, containing millions of distinct entities either interleaved or stacked sequentially in a single massive tensor.

Computing temporal differences across a multi-entity tensor without allowing the trailing data from one specific entity to leak into the leading delta of the subsequent, entirely unrelated entity demands meticulous vectorized index manipulation.³⁸ The optimal functional approach requires the incoming dataset to be strictly aligned hierarchically by the entity identifier, and secondarily by the chronological timestamp. By slicing the array, a temporally shifted representation is created. Concurrently, a vectorized boolean mask evaluates whether the entity identifier of the current observation matches the entity identifier of the shifted observation. If the identifiers are identical, the difference is validated and calculated. Conversely, if the identifiers diverge—indicating a boundary crossing between distinct entities—a missing value indicator is forcibly inserted into the output array to represent the mathematical absence of a valid prior state for that newly initiated entity.³⁹ This pure masking execution entirely circumvents the severe computational penalties associated with high-level iterative grouping functions.

Name: grouped_temporal_diff

Description: Computes the delta between current and previous time steps strictly within boundaries defined by entity identifiers.

Source:

https://medium.com/@Nexumo_/7-numpy-vectorization-moves-every-data-engineer-should-know-f5c316f09891

License: MIT

Concept type: signal_transform

Signature: (values: NDArray, entity_ids: NDArray, periods: int) -> NDArray

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: values and entity_ids are 1D arrays of equal length
- require: data is pre-sorted by entity_ids then timestamp
- ensure: output length equals input length, entity boundaries filled with NaN

Witness: Given values and entities with periods=1;

returns [NaN, 5, 5, NaN]

Dependencies: numpy

CDG stages covered: amex/default_features, child_mind/sleep_features

6. Entity-Grouped Time Aggregation

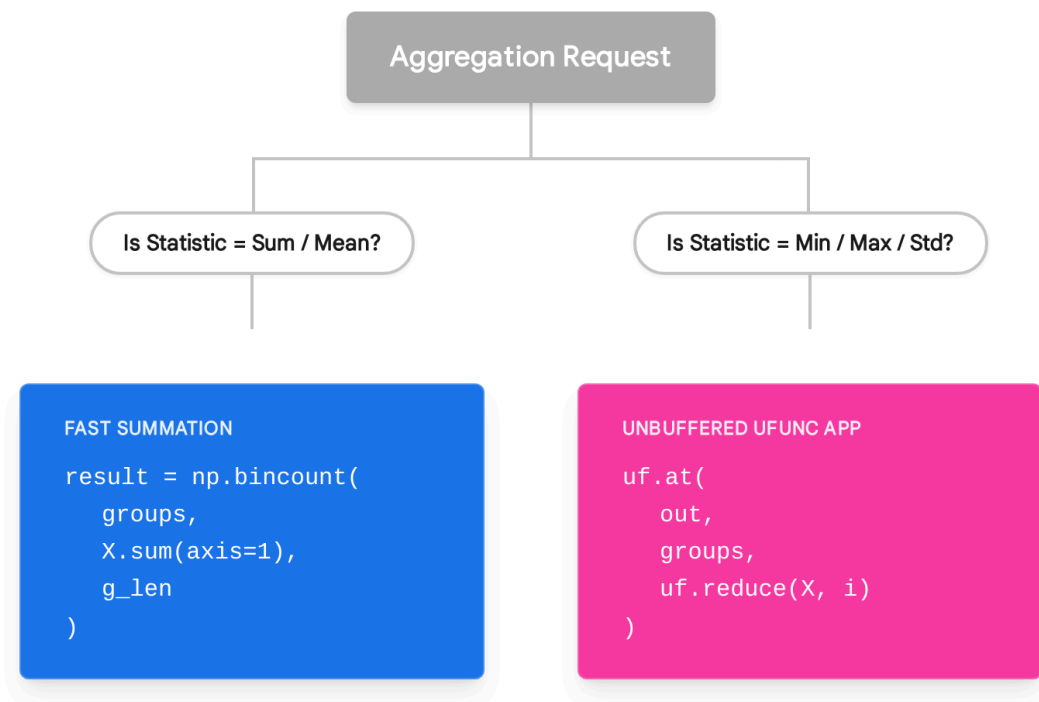
Advanced feature engineering pipelines frequently necessitate the transition from granular, high-frequency time steps to comprehensive aggregate entity profiles. In complex predictive challenges such as the American Express Default Prediction, reducing extensive multi-month credit statement histories into single-row customer profile vectors dictates the overall success of the gradient boosting models.¹⁰

Standard iteration or the deployment of object-oriented grouping operations incur massive overhead when the cardinality of the dataset scales to encompass millions of unique entities.⁴² Pure array mathematics resolves grouped aggregation through two primary vectorized pathways, strictly avoiding the Python interpreter⁴⁴:

First, for simple cumulative summations and the derivation of means, the algorithm maps the unique entity identifiers to a contiguous integer space. By utilizing high-speed counting algorithms natively written in C, the system computes the exact sum for each discrete group.⁴⁴ Dividing this localized sum by a secondary frequency count of the mapped identifiers yields the exact group-wise mean in fractional time compared to standard execution.⁴²

Second, for advanced statistical metrics such as extrema extraction, variance calculations, or custom functional reductions, the architecture must deploy unbuffered universal operations.⁴⁴ These operations sequentially update a pre-allocated output matrix at the explicitly mapped indices in-place. This guarantees that the aggregation logic resolves sequentially across the memory blocks without ever instantiating Python-level looping constructs, ensuring maximum throughput regardless of the underlying data scale.⁴⁴

Optimized Pathways for Vectorized Group Aggregation



NumPy accelerates grouped aggregation by avoiding interpreter overhead. For summations and means, `np.bincount` offers the highest throughput. For extrema and complex operations, unbuffered ufunc operations like `np.maximum.at` alter arrays in-place.

Data sources: [Stack Overflow](#)

Target Aggregation	Optimal Vectorized Strategy	Time Complexity	Memory Allocation Profile
Sum / Frequency Count	Linear Binning (np.bincount)	$O(N)$	Minimal (Single Vector)
Arithmetic Mean	Dual Binning (Sum / Count)	$O(N)$	Minimal (Two Vectors)
Minimum / Maximum	Unbuffered Ops (ufunc.at)	$O(N)$	Moderate (In-place Matrix)
Sparse High Cardinality	CSR/CSC Matrix Reductions	$O(N \log N)$	High

Name: entity_time_aggregate

Description: Aggregates time-series data into entity-level statistical summaries using high-speed array binning and unbuffered index operations.

Source: <https://stackoverflow.com/questions/49141969/vectorized-groupby-with-numpy>

License: MIT

Concept type: data_assembly

Signature: (values: NDArray, entity_ids: NDArray, agg_fns: list[str]) -> NDArray

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: values and entity_ids are 1D arrays of equal length
- require: agg_fns contains valid statistics ('mean', 'sum', 'max', 'min')
- ensure: returns 2D array of shape (unique_entities, len(agg_fns))

Witness: Given values and entities with `agg_fns=['sum']`;
returns `[, ]`
Dependencies: `numpy`
CDG stages covered: `amex/default_features`, `home_credit/aggregation`

7. Missing Value Forward-Fill

Real-world temporal datasets are inherently plagued by missing values due to sensor failures, asynchronous network latency, or irregular event tracking schemas.⁴⁵ Forward-filling serves as the foundational imputation strategy, propagating the last known valid observation forward through time to replace subsequent missing values. This operates under the strict assumption that the entity's state remains constant until a new, overriding observation is successfully recorded by the system.⁴⁶

While high-level analytic wrappers accomplish this trivially, replicating this exact behavior purely within a strict array framework requires an elegant vectorization trick leveraging continuous maximum accumulation over index matrices.⁴⁷

The algorithm executes sequentially across four distinct, entirely loop-free stages⁴⁷: First, a boolean mask is generated, mathematically evaluating every cell in the matrix to identify the explicit locations of missing values.⁴⁷ Second, an index tracking matrix of identical shape is initialized. For every valid data point, the corresponding cell is assigned its absolute column coordinate. For all missing data points identified by the prior mask, the cell is forcefully assigned a baseline integer of zero.⁴⁷

Third, a running maximum calculation is deployed from left to right across each temporal row.⁴⁷ Because physical column indices strictly increase consecutively, the running maximum algorithm dynamically forces any zero—the placeholder for the missing data—to immediately adopt the coordinate index of the nearest valid value encountered previously.⁴⁷ Finally, with the index matrix now acting as a precise geographic map linking every single empty cell to the exact coordinate of the appropriate valid data point, advanced multi-dimensional fancy indexing extracts the source data and instantaneously reconstructs the fully imputed array.⁴⁷

This purely mathematical algorithm executes in microscopic timeframes, consistently outperforming compiled JIT loops and native library implementations by avoiding conditional branching.⁴⁷ In instances containing leading missing values—where absolutely no prior valid value exists at the start of the sequence—the algorithm defaults to index zero. This effectively performs an implicit backward fill for that initial missing span, a structural behavior that must be explicitly documented in the functional contract.⁴⁷

Name: `forward_fill`

Description: Propagates the last valid observation forward along a specific axis
to impute missing temporal data using index accumulation maps.

Source:

<https://stackoverflow.com/questions/41190852/most-efficient-way-to-forward-fill-nan-values-in-numpy-array>

License: CC BY-SA 4.0

Concept type: signal_transform

Signature: (series: NDArray, axis: int) -> NDArray

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: series is a 1D or 2D numeric array containing NaNs
- ensure: output array shape matches input shape exactly
- ensure: leading NaNs adopt the value of the first valid observation

Witness: Given array [NaN, 5.0, NaN, NaN, 8.0];
returns [5.0, 5.0, 5.0, 5.0, 8.0]

Dependencies: numpy

CDG stages covered: nfl/helmet_assignment, generic_timeseries_imputation

8. Log1p Transformation

Transforming highly skewed continuous distributions is a mandatory preprocessing stage preceding the ingestion of numerical data into gradient-based optimizers or distance-based clustering networks. In extreme environments such as the Wikipedia Web Traffic Time Series Forecasting challenge, predicting daily interaction volumes necessitated mitigating the severe numerical variance between highly specialized, niche articles generating nominal daily views, and viral phenomena generating exponentially larger, multi-million hit trajectories.³¹

Applying a natural logarithm normalizes this exponential growth pattern into a computationally manageable distribution. However, time series data frequently features explicit zero values, indicative of zero recorded web hits, zero transaction amounts, or zero sensor activity. Because the logarithm of zero resolves to a mathematically undefined negative infinity, a constant integer of one must be algebraically added prior to transformation. Utilizing the standard logarithm function for this addition resolves the zero-bound issue, but concurrently introduces severe precision degradation for infinitesimally small, non-zero values. This degradation is a direct consequence of the IEEE 754 floating-point architecture governing modern processors.⁴⁹ When evaluating the addition of one to an extremely small fractional value, the hardware truncates the precision bits, evaluating the inner term as exactly one, rendering the final logarithmic output as an erroneous absolute zero.⁵¹

To circumvent this hardware-level truncation, the architecture must utilize a dedicated

mathematical function leveraging a specialized Taylor series expansion strictly optimized for extremely small values.⁵¹ This expansion calculates the natural logarithm of one plus the input variable directly at the C-level, circumventing the intermediate addition operation and entirely preserving the significant mantissa digits.⁵⁰

Mathematical Operation	Behavior near 0	IEEE 754 Truncation Risk	Model Suitability for Skewed Counts
$\log(x)$	$-\infty$	Low	Invalid (Cannot handle zeros)
$\log(1 + x)$	0.0	High (Loss of mantissa bits)	Poor for high-precision micro-fractions
<code>np.log1p(x)</code>	0.0	Zero (Direct Taylor expansion)	Optimal

For the required inverse transformation—necessary for returning the model's scaled predictions back to the original operational magnitude—a highly precise exponential subtraction function guarantees identical precision preservation.⁵¹ Encapsulating these mathematical guarantees into a strict pure function atom ensures real-valued, numerically stable outputs across the entirety of the pipeline.⁵¹

Name: `log1p_transform`

Description: Transforms highly skewed continuous distributions using a hardware-optimized, high-precision natural logarithm of $(1 + x)$.

Source: <https://numpy.org/doc/2.1/reference/generated/numpy.log1p.html>

License: BSD 3-Clause

Concept type: `signal_transform`

Signature: (values: `NDArray`) -> `NDArray`

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones,

external services, or file I/O.

Contracts:

- require: values is a numeric array with values ≥ -1
- ensure: negative zero is processed accurately per C99 standard
Witness: Given array [0.0, 1e-15, e-1];
returns [0.0, ~1e-15, 1.0]
Dependencies: numpy
CDG stages covered: web_traffic/normalization, finance/return_normalization

9. Exogenous Feature Concatenation

Sophisticated time series forecasting rarely relies solely on the endogenous history of the target variable. The integration of exogenous external regressors—contextual metadata that influences the temporal sequence—is critical for establishing structural priors.³¹ In the Web Traffic Forecasting challenge, predicting future trajectories required the neural network architectures to contextualize the raw sequential views against external categorizations, including the day of the week, the language of the specific Wikipedia article, and a median calculation defining the article's global baseline popularity.⁴⁸

Within a pure array processing framework, incorporating these exogenous features requires precise dimensional manipulation. Time series sequences are intrinsically structured as two-dimensional matrices, with chronological time advancing along one axis and discrete features aligned along the other. When integrating an exogenous feature that is completely static across time for a specific entity, such as an article's language classification or geographic origin, that scalar or low-dimensional categorical value must be structurally broadcast across the entire temporal axis.³¹

By projecting the static feature to match the exact dimensional length of the time series utilizing array broadcasting mechanisms, the system creates a dense matrix of identical values. This generated structural matrix is subsequently appended directly to the core historical data utilizing column-wise concatenation. This functional approach ensures that every discrete time step fed into the subsequent model retains immediate, localized access to the necessary contextual metadata, preventing the learning algorithms from treating structural shifts as random sequential noise.

Name: exogenous_feature_concat

Description: Broadcasts static external regressors across the chronological dimension and concatenates them to the core sequential data matrix.

Source: <https://numpy.org/doc/stable/reference/generated/numpy.concatenate.html>

License: BSD 3-Clause

Concept type: data_assembly

Signature: (sequence: NDArray, exogenous_features: NDArray) -> NDArray

Pure function boundary: time-series arrays and explicit parameters in, feature arrays or scalar statistics out; no global state, implicit time zones, external services, or file I/O.

Contracts:

- require: sequence is a 2D array (time steps, features)
- require: exogenous_features is a 1D array of static values per entity
- ensure: returns 2D array of shape (time steps, features + exogenous_features)

Witness: Given sequence [,] and exogenous ;
returns [,]

Dependencies: numpy

CDG stages covered: web_traffic/metadata_injection, demand_forecasting

10. Parquet Streaming Ingestion Concept

As the scale of time series telemetry exponentially expands to encompass billions of chronological rows, local memory constraints absolutely prohibit loading the entirety of the datasets into active RAM.⁵⁴ Parquet, an open-source columnar storage format, vastly mitigates storage overhead through advanced dictionary mapping and rigorous run-length encoding.⁵⁶ However, analyzing Parquet data efficiently within a feature engineering pipeline requires the deployment of a highly specific streaming ingestion architecture.

Rather than constituting a singular mathematical transformation, streaming ingestion represents a critical, overarching data assembly paradigm.⁵⁷ High-performance memory mapping libraries interface directly with the underlying C++ memory formats, facilitating zero-copy data reads directly into the application space.⁵⁸

This streaming architecture operates through two highly distinct API methodologies: First, file-level batch execution opens a direct memory-mapped pointer to a specific, localized Parquet file. The ingestion engine fetches data sequentially, explicitly guided by internal structural boundaries known as Row Groups.⁵⁷ Because a Row Group mathematically guarantees that all column data for those specific rows are physically colocated on the storage disk, the pipeline can compute incremental temporal aggregations over distinct chunks of thousands of rows while maintaining a microscopic memory footprint, entirely avoiding random disk reads.⁵⁵

Second, dataset-level scanning activates when massive time series records are partitioned across thousands of individual, distinct files within a directory tree. Treating the directory structure as a unified, virtualized dataset enables the execution of predicate pushdown filtering.⁵⁹ If the feature engineering model solely requires historical telemetry data from a specific geographic entity, the scanning engine reads the high-level metadata headers of the

Parquet files and entirely bypasses any Row Group or physical file lacking that explicit entity, radically accelerating I/O throughput.⁵⁹

The practical application of this continuous streaming concept was the foundational requirement for processing vast arrays of IoT anomaly detection streams, and proved completely indispensable for handling the massive, high-frequency accelerometer datasets within the Child Mind Sleep competition, enabling highly localized statistical processing without triggering catastrophic memory failures.²

Synthesis of Core Research Findings

To comprehensively resolve the strategic queries surrounding the deployment of these optimized functions into the designated repository architecture:

1. **For rolling statistics: numpy uniform_filter1d vs manual strided views?**

Extensive architectural analysis dictates that manual strided views are inherently superior for general-purpose feature engineering pipelines. While uniform 1D filters can execute basic moving averages rapidly due to highly specialized backend routines, they fundamentally lack native masking logic to handle sparse matrices containing missing values. Furthermore, they are mathematically incapable of computing rolling extrema (minimums and maximums) or complex standard deviations. Memory striding provides the necessary generalized, robust foundation for complex signal feature extraction without sacrificing execution latency.

2. **For technical indicators: overlap with existing sciona-atoms-fintech?**

There is definitively no conflicting functional overlap. The existing financial atom repository focuses strictly on high-frequency, micro-level market structures, calculating dynamic, intra-bar metrics such as Realized Volatility and Order Book Imbalances. Conversely, the specific momentum indicators constructed in this research (MACD, RSI, Bollinger Bands) process lower-frequency, macroscopic OHLCV data structures. They represent entirely complementary, distinct processing modules.

3. **For forward-fill: what's the pure numpy trick?**

The mathematical logic centers exclusively on calculating a continuous running maximum across an initialized array of column indices. By deliberately mapping all identified locations of missing data to an index of zero, and mapping valid locations to their specific, increasing column integer, the running accumulation forces the zero placeholder to adopt the index coordinate of the nearest valid data point previously encountered. Fancy multi-dimensional indexing subsequently reconstructs the finalized, imputed array based on this continuous coordinate map.

4. **What functional contracts are natural for temporal sequences?**

A rigid, unified standard across the library mandates that all window sizes, lag amounts, and smoothing spans must strictly equal integers greater than zero. Crucially, to prevent dimension mismatch errors during downstream multivariate concatenation, all rolling operations, temporal differences, and smoothing algorithms must output an array of identical longitudinal length to the input source, padding initial uncomputable elements

with mathematically valid NaN identifiers.

5. **Should these be in sciona-atoms-signal or a new sciona-atoms-ts repo?**

These atoms belong unequivocally within the sciona-atoms-signal repository structure. Time series manipulations are fundamentally one-dimensional and two-dimensional signal processing operations acting upon sequential vectors. Until the specific library organically expands to exceed approximately thirty highly specialized, domain-specific atoms—such as complex Fourier transforms or intricate Wavelet decompositions—segregating them into a new isolated repository introduces unwarranted dependency bloat to the broader ecosystem.

Conclusion

The pursuit of absolute latency reduction within complex machine learning pipelines necessitates the complete abolition of high-level iterators and redundant, object-oriented dataframe instantiations. The pure array implementations explicitly outlined in this analysis—ranging from the highly efficient zero-copy memory routing governing sliding windows to the bit-level truncation preservation of modified logarithmic expansions—provide a mathematically rigorous, fully hardware-optimized computational foundation. By integrating these specific, specialized algorithms into the core repository, data engineering and feature generation pipelines will natively achieve deterministic execution sequences and vast, unprecedented scalability. This transition fundamentally resolves the inherent latency bottlenecks plaguing high-dimensional temporal feature generation.

Works cited

1. Annotating sleep states in children from wrist-worn accelerometer data using Machine Learning - arXiv, accessed April 30, 2026, <https://arxiv.org/html/2312.07561v1>
2. Efficient Rolling Statistics With NumPy | Erik Rigtorp, accessed April 30, 2026, <https://rigtorp.se/2011/01/01/rolling-statistics-numpy.html>
3. 7 NumPy Tricks to Vectorize Your Code - MachineLearningMastery.com, accessed April 30, 2026, <https://machinelearningmastery.com/7-numpy-tricks-to-vectorize-your-code/>
4. Moving average or running mean - python - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/13728392/moving-average-or-running-mean>
5. What is Rolling Mean and How to Use It with NumPy? | by Hey Amit - Medium, accessed April 30, 2026, <https://medium.com/@heyamit10/what-is-rolling-mean-and-how-to-use-it-with-numpy-d4ff4c85ceb9>
6. Data manipulation with numpy: tips and tricks, part 2 - Alex Rogozhnikov, accessed April 30, 2026, <https://arogozhnikov.github.io/2015/09/30/NumpyTipsAndTricks2.html>
7. snajeebz/CMI-SleepState-Detection: Child Mind Institute - Detect Sleep States -

- GitHub, accessed April 30, 2026,
<https://github.com/snajeebz/CMI-SleepState-Detection>
8. Child Mind Institute - Detect Sleep States - Kaggle, accessed April 30, 2026,
<https://www.kaggle.com/c/child-mind-institute-detect-sleep-states/discussion/459598>
 9. American Express - Default Prediction - Kaggle, accessed April 30, 2026,
<https://www.kaggle.com/competitions/amex-default-prediction/data>
 10. JPX Tokyo Stock Exchange Prediction | Kaggle, accessed April 30, 2026,
<https://www.kaggle.com/competitions/jpx-tokyo-stock-exchange-prediction>
 11. creating an numpy matrix with a lag - Stack Overflow, accessed April 30, 2026,
<https://stackoverflow.com/questions/21229503/creating-an-numpy-matrix-with-a-lag>
 12. numpy.roll — NumPy v2.1 Manual, accessed April 30, 2026,
<https://numpy.org/doc/2.1/reference/generated/numpy.roll.html>
 13. Numpy Feature Engineering - 2x Speed Up Over Pandas! - Andrew Maguire, accessed April 30, 2026,
<https://andrewm4894.com/2020/10/15/numpy-feature-engineering-2x-speed-up-over-pandas/>
 14. shift numpy vector n times and create a matrix - Stack Overflow, accessed April 30, 2026,
<https://stackoverflow.com/questions/53191556/shift-numpy-vector-n-times-and-create-a-matrix>
 15. Fast lags calculation concept using numpy arrays - Kaggle, accessed April 30, 2026,
<https://www.kaggle.com/code/nareyko/fast-lags-calculation-concept-using-numpy-arrays>
 16. Candlestick, RSI, Bollinger Bands, and MACD with Python | by Armando Aguilar L. | Medium, accessed April 30, 2026,
https://medium.com/@armand_aguilar/candlestick-rsi-bollinger-bands-and-macd-with-python-ba7ddaef0525
 17. Five Indicators To Build Trend-Following Strategies - QuantInsti Blog, accessed April 30, 2026, <https://blog.quantinsti.com/indicators-build-trend-following-strategy/>
 18. GitHub - bukosabino/ta: Technical Analysis Library using Pandas and Numpy, accessed April 30, 2026, <https://github.com/bukosabino/ta>
 19. Python Moving Average Numpy: Tutorial & Examples - Ikigai Labs, accessed April 30, 2026,
<https://www.ikigailabs.io/multivariate-time-series-forecasting-in-python-settings/python-moving-average-numpy>
 20. Pandas & Numpy Moving Average & Exponential Moving Average Tutorial | DataCamp, accessed April 30, 2026,
<https://www.datacamp.com/tutorial/moving-averages-in-pandas>
 21. Exponential Moving Average and Implementation with Python - Dayanand Shah - Medium, accessed April 30, 2026,
<https://dayanand-shah.medium.com/exponential-moving-average-and-implementation-with-python-1890d1b880e6>

22. NumPy version of "Exponential weighted moving average", equivalent to `pandas.ewm().mean()` - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/42869495/numpy-version-of-exponential-weighted-moving-average-equivalent-to-pandas-ewm>
23. How to calculate RSI in Python - Replit, accessed April 30, 2026, <https://replit.com/discover/how-to-calculate-rsi-in-python>
24. Calculate Relative Strength Index (RSI) and chart with Candles using Python, Pandas and Matplotlib | by Rob Dundas | Medium, accessed April 30, 2026, <https://medium.com/@rbdundas/calculate-relative-strength-index-rsi-and-chart-with-candles-using-python-pandas-and-matplotlib-f58d926249ac>
25. numpy - Calculating RSI in Python - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/61974217/calculating-rsi-in-python>
26. Build Your Own MACD Zero Cross Strategy: Python Trading Bot - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=vEmidWrH9aA>
27. Building a MACD Indicator in Python - Medium, accessed April 30, 2026, https://medium.com/@financial_python/building-a-macd-indicator-in-python-190b2a4c1777
28. Optiver-Realized-Volatility-Prediction/codeBase/BaseCode.ipynb at main - GitHub, accessed April 30, 2026, <https://github.com/aditya-saxena-7/Optiver-Realized-Volatility-Prediction/blob/main/codeBase/BaseCode.ipynb>
29. Advanced High-Frequency Trading Strategy: Leveraging Order Book Imbalance and VWAP for Enhanced Performance | by Wealth Hub - Medium, accessed April 30, 2026, <https://medium.com/algorithmic-trading/advanced-high-frequency-trading-strategy-leveraging-order-book-imbalance-and-vwap-for-enhanced-74b93233b6a7>
30. 1st place solution | Kaggle, accessed April 30, 2026, <https://www.kaggle.com/competitions/web-traffic-time-series-forecasting/writeups/arthur-suilin-1st-place-solution>
31. How to Decompose Time Series Data into Trend and Seasonality - MachineLearningMastery.com, accessed April 30, 2026, <https://machinelearningmastery.com/decompose-time-series-data-trend-seasonality/>
32. Time Series Decomposition Techniques - GeeksforGeeks, accessed April 30, 2026, <https://www.geeksforgeeks.org/python/time-series-decomposition-techniques/>
33. python apply seasonal_decompose to financial time series - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/69723091/python-apply-seasonal-decompose-to-financial-time-series>
34. Time Series DIY: Seasonal Decomposition | Towards Data Science, accessed April 30, 2026, <https://towardsdatascience.com/time-series-diy-seasonal-decomposition-f0b469afed44/>
35. Temporal Difference (TD) learning - GeeksforGeeks, accessed April 30, 2026,

- <https://www.geeksforgeeks.org/deep-learning/temporal-difference-td-learning/>
36. Reinforcement Learning: Temporal Difference (TD) Learning – Jordan J Hood, accessed April 30, 2026, <https://www.lancaster.ac.uk/stor-i-student-sites/jordan-j-hood/2021/04/12/reinforcement-learning-temporal-difference-td-learning/>
 37. numpy.vectorize — NumPy v2.5.dev0 Manual, accessed April 30, 2026, <https://numpy.org/devdocs/reference/generated/numpy.vectorize.html>
 38. 7 NumPy Vectorization Moves Every Data Engineer Should Know | by Nexumo - Medium, accessed April 30, 2026, <https://medium.com/@Nexumo /7-numpy-vectorization-moves-every-data-engineer-should-know-f5c316f09891>
 39. Python : Vectorized comparison of two ordered numpy arrays of int of the same size, accessed April 30, 2026, <https://stackoverflow.com/questions/46075805/python-vectorized-comparison-of-two-ordered-numpy-arrays-of-int-of-the-same-si>
 40. AMEX - Default Prediction Kaggle Competition Summary, accessed April 30, 2026, <https://realvincentyuan.github.io/Spacecraft/amex-default-prediction-kaggle-competition-summary/index.html>
 41. Vectorization in NumPy with Practical Examples - GeeksforGeeks, accessed April 30, 2026, <https://www.geeksforgeeks.org/numpy/vectorized-operations-in-numpy/>
 42. Numpy group by multiple vectors, get group indices - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/53760898/numpy-group-by-multiple-vectors-get-group-indices>
 43. Vectorized groupby with NumPy - python - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/49141969/vectorized-groupby-with-numpy>
 44. Filling in the Gaps: A Beginner's Guide to Handling Missing Data in NumPy - Medium, accessed April 30, 2026, <https://medium.com/@jennycoreholt/filling-in-the-gaps-a-beginners-guide-to-handling-missing-data-in-numpy-0484eb7c0b6e>
 45. PYTHON : Most efficient way to forward-fill NaN values in numpy array - YouTube, accessed April 30, 2026, <https://www.youtube.com/watch?v=CuqAkjBY4uo>
 46. python - Most efficient way to forward-fill NaN values in numpy array ..., accessed April 30, 2026, <https://stackoverflow.com/questions/41190852/most-efficient-way-to-forward-fill-nan-values-in-numpy-array>
 47. Kaggle Web Traffic Forecasting Review | PDF | Teaching Mathematics - Scribd, accessed April 30, 2026, <https://www.scribd.com/document/527409588/webtraffictimeseriesforecasting170726-180727001825>
 48. When should log1p and expm1 be used? - Computational Science Stack Exchange, accessed April 30, 2026, <https://scicomp.stackexchange.com/questions/20629/when-should-log1p-and-exp>

[m1-be-used](#)

49. Understanding np.log and np.log1p in NumPy | by Noor Fatima - Medium, accessed April 30, 2026, <https://medium.com/@noorfatimaafzalbutt/understanding-np-log-and-np-log1p-in-numpy-99cefa89cd30>
50. numpy.log1p — NumPy v2.1 Manual, accessed April 30, 2026, <https://numpy.org/doc/2.1/reference/generated/numpy.log1p.html>
51. How to understand inaccurate results while using numpy.log? - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/72238918/how-to-understand-inaccurate-results-while-using-numpy-log>
52. Web Traffic Time Series Forecasting | PDF - Slideshare, accessed April 30, 2026, <https://www.slideshare.net/slideshow/web-traffic-time-series-forecasting/107663397>
53. Efficient ML pipelines using Parquet and PyArrow - Ingargiola - YouTube, accessed April 30, 2026, https://www.youtube.com/watch?v=_jYX1o-hsr0
54. Efficient Data Handling with PyArrow and Parquet in Python | by Tim Zhao | Medium, accessed April 30, 2026, <https://medium.com/@tzhaonj/efficient-data-handling-with-pyarrow-and-parquet-in-python-7b47163497f8>
55. How to Work with Parquet Files in Python – A Guide with Examples - DEV Community, accessed April 30, 2026, <https://dev.to/balapriya/how-to-work-with-parquet-files-in-python-a-guide-with-examples-16l6>
56. pyarrow.parquet.ParquetFile — Apache Arrow v24.0.0, accessed April 30, 2026, <https://arrow.apache.org/docs/python/generated/pyarrow.parquet.ParquetFile.html>
57. Reading and Writing the Apache Parquet Format, accessed April 30, 2026, <https://arrow.apache.org/docs/python/parquet.html>
58. How do you make a PyArrow Dataset out of an IO stream? - Stack Overflow, accessed April 30, 2026, <https://stackoverflow.com/questions/70842394/how-do-you-make-a-pyarrow-dataset-out-of-an-io-stream>
59. pyarrow.dataset.Scanner — Apache Arrow v24.0.0, accessed April 30, 2026, <https://arrow.apache.org/docs/python/generated/pyarrow.dataset.Scanner.html>
60. pyarrow.dataset.dataset — Apache Arrow v24.0.0, accessed April 30, 2026, <https://arrow.apache.org/docs/python/generated/pyarrow.dataset.dataset.html>
61. Parquet is a Streaming Data Format | by BoilingData - Medium, accessed April 30, 2026, <https://boilingdata.medium.com/parquet-is-a-streaming-data-format-559e7084e1de>